Based on that, you enter either git bisect good or git bisect bad. (If you really can't tell, you can do git bisect skip.)

Git now has enough information to know that the bad change was in one half of the commits. If you said the commit was bad, then the change was in the first half of the commits; if you said it was good, then the change had to come after. Git splits the narrowed-down list of commits in half and checks out the middle commit.

You repeat your checks until eventually you can isolate a change where the beginning state is good and the end state is bad. Ideally, inspecting the commit list of the commit with that change will give you a hint as to what's causing the behavior, since one of those changes is likely the cause. This process can save you hours.

Furthermore, if you can encapsulate whatever test you're running against your codebase in a script that follows the Unix convention of returning a nonzero value on failure, you can pass that script to git bisect and Git will automatically do the good/bad thing for you based on the result of the script. The syntax is of the following form:

```
$ git bisect run my_script
```

I note in passing that whichever tool you're using to run your tests has this behavior, so git bisect run rspec should work, though it'll probably be a little on the slow side.

For the full git bisect effect, a few things need to be true:

- The problem needs to have been caused by a code change, not a change in your environment.
- You need to be able to reliably trigger the problem.
- It helps if your commits are relatively small and if the system is in a loadable and executable state after each one.

That said, when this works, it can work big.

RSpec or Minitest Bisect

Minitest and RSpec both have their own version of bisect, which solves a slightly different problem. Sometimes you'll have a test failure caused by the order in which the tests are run. This usually means the tests aren't completely isolated and one test is changing global state in a way that breaks another test. This can be very difficult to reproduce, especially if the test suite is long.